

InsightStream

AI-Native UX Spec & Vibe Map

Confidence-State Mapping • Verification Center • Perceived Latency • Search UX • Export Flow

Design Philosophy: Move the PM from Synthesizer to Editor

Table of Contents

0. Design Philosophy: Synthesizer → Editor	3
1. Confidence-State UI Mapping.....	4
1.1 High Confidence (9–10): The “Verified” State	4
1.2 Medium Confidence (5–8): The “Verify” State	5
1.3 Low Confidence (1–4): The “Possible Insight” State	6
1.4 Confidence-State Logic Flow.....	6
2. The Verification Center: Interaction Model.....	7
2.1 Split-Pane Architecture.....	7
2.2 Deep-Link UX: Quote-to-Transcript Navigation	7
Current Implementation	7
Target Implementation (v2)	7
2.3 Card Interaction Patterns.....	8
3. Perceived Latency & The “Wait Vibe”	9
3.1 The 45-Second Wait Vibe Map	9
3.2 Current vs. Target Implementation	9
3.3 Why Not Streaming?	10
4. The “Missing Something?” Search UX.....	11
4.1 Interaction Flow	11
4.2 UI State Transitions	11
4.3 Implicit Feedback Signal.....	11
5. The Export Flow: From AI Extraction to Engineering Execution.....	12
5.1 Current Export: CSV	12
5.2 Target Export: Jira / Linear / Productboard Integration	12
Jira Integration Spec	12
5.3 Export Interaction Design.....	13
6. AI Feedback Signals: How the System Learns from User Behavior.....	14
7. Accessibility and Inclusive Design.....	15

0. Design Philosophy: Synthesizer → Editor

The traditional PM workflow for interview analysis is: read transcript → highlight quotes → tag themes → write summaries → score priorities. The PM is the synthesizer. This takes 4 hours per interview.

Insight Stream's UX goal is to invert this relationship. The AI does the synthesis. The PM becomes the editor: reviewing, correcting, approving, and exporting. This is a fundamentally different interaction pattern.

Dimension	PM as Synthesizer (Before)	PM as Editor (InsightStream)
Primary action	Create insights from scratch	Review, approve, or reject pre-extracted insights
Cognitive load	High: reading, interpreting, writing simultaneously	Low: binary decisions (yes/no) on pre-formed outputs
Time per interview	4 hours	10–15 minutes (review only)
Error mode	Missed insights (recall failure)	Accepted hallucinations (precision failure)
Trust model	Self-trust (I read it myself)	Evidence-trust (AI shows me the exact quote)

The UX implication: Every design decision must reduce the cognitive cost of editing while maximizing the PM's ability to spot errors. The UI is not a dashboard. It is a review workflow.

1. Confidence-State UI Mapping

The AI's confidence score (1–10) is not just a number. It is a signal that should change how the interface behaves. High-confidence insights should feel settled. Low-confidence insights should feel provisional. The UI must communicate this distinction through visual hierarchy, not just a number.

1.1 High Confidence (9–10): The “Verified” State

VIBE: Certainty. Calm. This feels done.

UI Element	Specification
Card border	Left border: 4px solid #1E8449 (green). Subtle. Not overwhelming.
Confidence badge	Green pill badge: "9/10 • Verified" or "10/10 • Explicit Request". Uses the rubric language.
Approve checkbox	Pre-checked (default: approved). The PM only needs to act if they disagree.
Supporting quote	Displayed in standard weight. No special highlighting — the green border already signals confidence.
Card opacity	100%. Full presence. This card belongs here.
Interaction expectation	Scan and move on. The PM should spend < 5 seconds per high-confidence card.

Design rationale: The user said “Please add dark mode.” The AI extracted “User wants dark mode.” Confidence = 10. There is nothing to verify. The UI should communicate: “this one is handled.”

1.2 Medium Confidence (5–8): The “Verify” State

VIBE: Useful but uncertain. Worth checking. Your call.

UI Element	Specification
Card border	Left border: 4px solid #D4AC0D (amber). Attention-drawing but not alarming.
Confidence badge	Amber pill: "6/10 • Implicit" or "7/10 • Inferred from Context". Honest about the inference gap.
CTA button	"Verify Source  button below the insight. Clicking scrolls the transcript pane to the exact quote location.
Supporting quote	Displayed with amber background highlight on the quote text. Visual signal: "look at this more carefully."
Approve checkbox	Pre-checked but with amber dot indicator. The PM is nudged to consciously confirm rather than auto-approve.
Card opacity	100%. Still fully present — this is a real insight, just needs verification.
Interaction expectation	15–30 seconds. Read the quote, check context, decide. The "Verify Source" button is the key interaction.

Design rationale: The user said “I end up having to message her on Slack separately.” The AI inferred “User wants @mention notifications.” Confidence = 7. The inference is reasonable but not explicit. The PM should check.

1.3 Low Confidence (1–4): The “Possible Insight” State

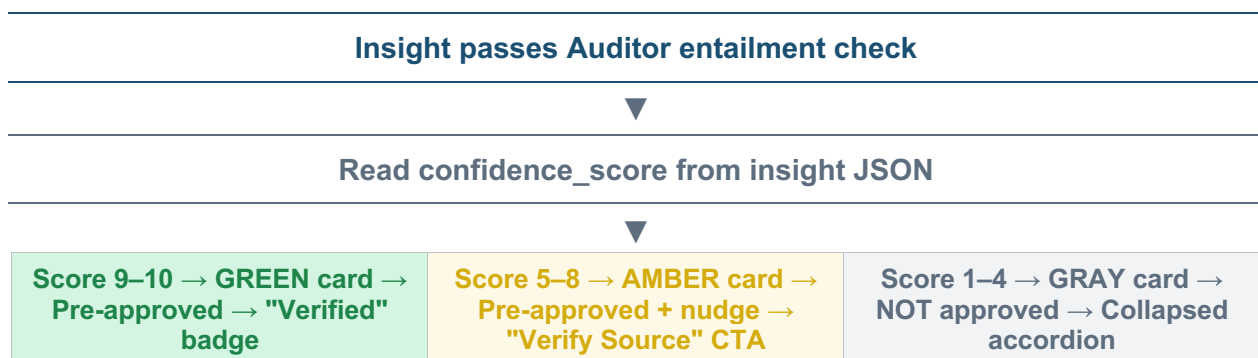
VIBE: Humble. Tentative. The AI is guessing. You decide.

UI Element	Specification
Card border	Left border: 4px solid #BDC3C7 (light gray). Deliberately understated. This card is provisional.
Confidence badge	Gray pill: "3/10 • Possible Insight" or "2/10 • Vague Signal". Framing: possibility, not conclusion.
Card header	Prefixed with "Possible: " in gray text. Example: "Possible: User may want calendar integration."
Approve checkbox	NOT pre-checked. Default: excluded from export. The PM must actively choose to include it.
Supporting quote	Displayed in gray italic text. Visually receded. The quote is there but doesn't dominate.
Card opacity	70%. Visually faded compared to high/medium cards. Peripheral, not primary.
Collapse behavior	Collapsed by default in a "Show N possible insights" accordion. Only expands on click.
Interaction expectation	Optional. Most PMs will skip these. Power users will expand and occasionally promote one to approved.

Design rationale: The user said “It’s not ideal, to be honest.” The AI extracted a vague complaint with confidence = 3. This might be a real pain point expressed through British understatement, or it might be genuinely minor. The UI says: “here it is, but we’re not sure. You decide.”

1.4 Confidence-State Logic Flow

The following diagram shows how the UI renders each insight based on its confidence score after passing the Auditor:



2. The Verification Center: Interaction Model

2.1 Split-Pane Architecture

The Verification Center uses a conceptual split-pane layout. The current Streamlit implementation renders these as stacked sections, but the mental model is two synchronized views:

Left Pane: Structured Insights	Right Pane: Raw Transcript
Pain Point and Feature Request cards arranged in 2-column grid. Each card shows: confidence badge, issue/need description, impact/solution, verbatim quote (highlighted), source_id, and approve/reject checkbox.	Full transcript text with speaker labels. The "Missing Something?" search bar is positioned at the top. Search results appear inline. The transcript serves as the ground truth reference.

2.2 Deep-Link UX: Quote-to-Transcript Navigation

When the PM clicks "Verify Source" on an insight card, the UI should scroll the transcript pane to the exact location of the supporting quote and highlight it. This is the core trust interaction.

Current Implementation

The current Streamlit build displays the supporting quote directly on each insight card. The transcript is available in an expandable section above. There is no automatic scroll-to-quote functionality.

Target Implementation (v2)

1. Each supporting_quote is matched to its position in the raw transcript using string search (the same _get_segment_around_quote logic used by the Auditor).
2. Clicking "Verify Source  on an insight card scrolls the transcript view to the matched position and applies a yellow background highlight to the exact quote span.
3. An 800-character context window is displayed around the quote (400 chars before, 400 after) so the PM can read surrounding context without scanning the entire transcript.
4. A "Back to insights" button returns focus to the card grid.
5. If the quote is not found in the transcript (possible LLM paraphrasing), a red warning appears: "Exact quote not found in transcript. The AI may have paraphrased."

2.3 Card Interaction Patterns

User Action	UI Response	Data Signal	Feedback Value
Check “Approve for Backlog”	Checkbox fills green. Card border pulses briefly.	APPROVE signal logged	Positive training pair
Uncheck “Approve for Backlog”	Checkbox clears. Card fades to 60% opacity.	REJECT signal logged	Negative training pair (highest value)
Click “Verify Source”	Transcript scrolls to quote. Quote highlighted.	VERIFY action logged	Indicates medium-confidence insight needed checking
Expand collapsed low-confidence card	Accordion expands. Card renders at 70% opacity.	EXPAND action logged	User is looking for edge cases (power user signal)
Hover over confidence badge	Tooltip shows rubric definition for that score range	No signal	Educational — builds PM's mental model of scoring

3. Perceived Latency & The “Wait Vibe”

InsightStream’s pipeline takes 15–45 seconds. This is too long for a spinner, too short for a progress bar, and completely wrong for streaming (structured JSON output cannot be streamed into card layouts). The solution is a dynamic progress indicator that narrates what the system is doing.

3.1 The 45-Second Wait Vibe Map

Seconds	Pipeline Phase	User Sees	Emotional Vibe
0–2s	Upload + Preprocessing	File name appears. Transcript preview available in expander.	Instant feedback. "It received my file."
2–5s	Router + Global Context	Spinner: "Reading transcript and building context..."	The system is thinking. Normal.
5–15s	Extractor (parallel chunks)	Spinner: "Extracting insights from [N] transcript segments..."	Active progress. Work is happening.
15–20s	Reducer + Formatter + Dedup	Spinner: "Merging and deduplicating [X] raw insights..."	Momentum. Getting closer.
20–35s	Auditor (entailment)	Spinner: "Verifying each insight against the transcript..."	The critical phase. Trust is being built.
35–40s	Synthesizer	Spinner: "Synthesizing core themes from verified insights..."	Almost there.
40–45s	Render	Toast: "Analysis complete!" Full Verification Center appears.	Done. Moment of reveal.

3.2 Current vs. Target Implementation

Current (v1): Single spinner with static message: “Processing {filename}...” for the entire duration. Works but provides no sense of progress.

Target (v2): Multi-phase spinner that updates its message at each pipeline stage. LangGraph provides node-level callbacks that can trigger UI updates. Each phase message includes the node name and a count (e.g., “3 segments” or “7 raw insights”) to show concrete progress.

Future (v3): Progressive card rendering. As each Extractor chunk completes, its insights appear immediately in a “Pending verification” state (grayed out). When the Auditor verifies each one, the card transitions to its confidence-state color. The user watches the results build in real time.

3.3 Why Not Streaming?

InsightStream's output is structured data (JSON objects rendered as cards with metrics, checkboxes, and colored borders). Streaming is designed for prose — one token at a time appearing in a text block.

Attempting to stream structured output creates three problems:

- Incomplete JSON cannot be rendered as a card. A half-formed object crashes the UI or displays gibberish.
- Card layouts shift as new data arrives, causing the visual “jumping” that users find disorienting.
- The confidence-state visual treatment (green/amber/gray) cannot be applied until the full insight is available.

The correct pattern for structured output is: wait silently (with narrated progress), then render everything at once. This is the approach used by Figma's AI features, Notion AI, and other tools that produce structured rather than prose output.

4. The “Missing Something?” Search UX

The search bar is InsightStream’s safety net. It catches everything the automated pipeline misses. But it also serves a deeper UX purpose: it gives the PM permission to trust the AI. If the PM knows they can always search for anything the AI missed, they feel less anxious about approving the automated results.

4.1 Interaction Flow

The search bar appears directly below the Core Themes section, above the Pain Points and Feature Requests. Position is intentional: the PM reviews themes first, then searches before diving into individual insights.

1. The placeholder text reads: “e.g. What did they say about login speed?” — modeling the expected query format.
2. On submit, a spinner appears: “Searching transcript...” (typically 2–5 seconds for the RAG query).
3. The answer appears as a markdown block directly below the search bar. It is clearly labeled as AI-generated and includes relevant quotes from the transcript.
4. If nothing relevant is found: “The transcript does not contain relevant information about [topic].” This is a definitive negative, not a hedge.

4.2 UI State Transitions

State	UI Appearance	User Expectation
Idle (no query)	Empty search bar with placeholder text. Muted, non-intrusive.	"I can search if I need to. But I might not need to."
Searching	Spinner replaces search bar briefly: "Searching transcript..."	"It's looking through the full transcript for me."
Result found	Answer block appears with markdown formatting. Relevant quotes in italics.	"Found it. Now I can decide if this should have been extracted."
No result	Caption: "The transcript does not contain relevant information about [topic]."	"The pipeline didn't miss it — it's just not in the transcript."

4.3 Implicit Feedback Signal

Every search query is a coverage gap signal. If the PM searches for “What did they say about onboarding?” and finds relevant content, that means the pipeline missed an insight about onboarding. The query text itself reveals what the user expected but didn’t get.

In the future telemetry system, search queries are logged as `COVERAGE_GAP` events with the query text. Over time, frequently searched topics that the pipeline consistently misses become candidates for prompt optimization.

5. The Export Flow: From AI Extraction to Engineering Execution

5.1 Current Export: CSV

The current export produces a CSV file with five columns: Category (Pain Point / Feature Request), Core Issue/Need, Impact/Solution, Supporting Quote, and Confidence Score. Only insights with “Approve for Backlog” checked are included.

This CSV is the deterministic output of the human review process. Every row was seen, evaluated, and approved by the PM. It is the correction-verified artifact that bridges AI extraction and product execution.

5.2 Target Export: Jira / Linear / Productboard Integration

The CSV-to-Jira gap is the biggest friction point in the current workflow. The PM exports a CSV, then manually creates Jira tickets from each row. The target integration eliminates this step.

Jira Integration Spec

Field	Mapping
Issue Type	Pain Point → Bug or Improvement. Feature Request → Story.
Summary	user_need (for Feature Requests) or issue_description (for Pain Points). Truncated to 255 chars.
Description	Full insight: need/issue + solution/impact + verbatim supporting quote + confidence score + source attribution.
Priority	Mapped from confidence_score: 9–10 → High, 7–8 → Medium, 4–6 → Low, 1–3 → Lowest.
Labels	Auto-tagged: "insightstream", "user-research", and the core theme name (e.g., "theme:performance").
Custom field: Source	"InsightStream v2.0 — Interview: {transcript_name} — Confidence: {score}/10"
Custom field: Evidence	The verbatim supporting_quote. Formatted as a Jira quote block.

5.3 Export Interaction Design

Step	User Action	System Response	Safeguard
1. Review	PM finishes reviewing all insight cards	Export section becomes active	Export button disabled until at least 1 insight is approved
2. Choose format	PM selects: CSV, Jira, Linear, or Productboard	Format-specific options appear (e.g., Jira project selector)	Jira/Linear require OAuth connection (setup once)
3. Preview	PM clicks "Preview Export"	Table shows exactly what will be created: N tickets, mapped fields	No ticket is created until explicit confirmation
4. Confirm	PM clicks "Create N Tickets"	Progress bar: "Creating ticket 1 of N..."	Each ticket links back to InsightStream session ID for audit trail
5. Done	PM sees confirmation with links to created tickets	"Created 7 tickets in PROJECT. View in Jira 	Correction log records which insights were exported to which tickets

Key safeguard: The preview step is non-negotiable. A PM accidentally creating 15 Jira tickets from unreviewed AI output would erode trust in the entire system. The preview forces a final conscious confirmation before anything hits the engineering backlog.

6. AI Feedback Signals: How the System Learns from User Behavior

Every interaction in the Verification Center generates a signal that can improve future extractions. The PM never “trains the AI” explicitly. They just do their job. The training data is a byproduct of normal workflow usage.

User Action	Signal Type	Data Captured	Training Use	Priority
Approves insight	POSITIVE	(transcript_segment, insight) → GOOD	Positive pair for DPO	Medium
Rejects insight	NEGATIVE	(transcript_segment, insight) → BAD	Negative pair for DPO	Highest
Uses "Verify Source"	UNCERTAINTY	Insight ID flagged as "needed verification"	Calibration signal	Low
Searches transcript	COVERAGE GAP	Query text + presence/absence of result	Recall improvement	High
Swaps speakers	DIARIZATION ERROR	Audio hash + swap event	Preprocessing improvement	Medium
Edits insight before export (v2)	CORRECTION	(original_AI_output, human_corrected_output)	DPO preferred/rejected pair	Highest
Adds manual insight (v2)	MISSED	New insight not in AI output + transcript context	Direct recall supervision	High
Time spent per card (v3)	DIFFICULTY	Dwell time per insight card	Weight harder cases in training	Low
Export count per session	UTILITY	N approved / N total insights	Session-level quality metric	Low

Design principle: The highest-value signals (REJECT and CORRECTION) come from normal workflow actions, not special feedback forms. The PM is doing their job. The system is learning in the background. This is the invisible flywheel.

7. Accessibility and Inclusive Design

The Verification Center must be usable by PMs with varying abilities. The following requirements apply to the confidence-state visual system:

- Color is never the sole indicator. Every confidence state uses color + text label + icon. Green border + "Verified" badge + checkmark icon. Amber border + "Implicit" badge + question mark icon. Gray border + "Possible" text + dash icon. Color-blind users can distinguish states without color.
- All interactive elements are keyboard-navigable. Tab moves between insight cards. Enter toggles the approve checkbox. Space expands collapsed cards. Arrow keys navigate within the transcript pane.
- Screen reader support. Each card announces: "Feature request, confidence 8 out of 10, implicit. User need: [text]. Approved for backlog: checked." The confidence badge tooltip content is included in the aria-label.
- Minimum contrast ratios. All text meets WCAG 2.1 AA (4.5:1 for normal text, 3:1 for large text). The muted gray of low-confidence cards (70% opacity) still meets contrast requirements against the dark background.
- Reduced motion preference. If the user's OS is set to "reduce motion," the card border pulse on approval and the scroll-to-quote animation are disabled. State changes are instant.